Multi-Course Attendance Management Portal

Module B: Optimisation Implementation Report

CS 432 — Databases

Assignment 2 · Track 1 · Semester II, 2025–26

Name	Roll Number
Rishabh Jogani	23110276
Sidhharth Rajandekar	23110310
Tejas Lohia	23110335
Umang Shikarvar	23110301
Zainab Kapadia	23110373

Repository: GitHub Repo Link

Video Link: Video Link

Contents

1 Schema Design

1.1 Design Philosophy

The schema is divided into two categories to avoid duplicating credentials or identity data inside project-specific tables.

- **Core system tables:** `users`, `sessions`, `user_profiles` — manage authentication and identity only.
- **Project-specific tables:** all attendance, course, and correction tables — reference `users.id` via foreign keys but contain no passwords or session tokens.

Foreign key enforcement is activated on every connection: `PRAGMA foreign_keys = ON`.

1.2 Core System Tables

```
CREATE TABLE IF NOT EXISTS users (
    user_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    username   TEXT    UNIQUE NOT NULL,
    pwd_hash   TEXT    NOT NULL,
    role       TEXT    CHECK(role IN ('admin','instructor','student',
       'ta','dean')) NOT NULL,
    last_login TEXT
);

CREATE TABLE IF NOT EXISTS user_profiles (
    user_id     INTEGER PRIMARY KEY REFERENCES users(user_id) ON
       DELETE CASCADE,
    roll_no     TEXT,
    program     TEXT,
    batch       TEXT,
    department  TEXT,
    designation TEXT
);

CREATE TABLE IF NOT EXISTS sessions (
    session_id TEXT    PRIMARY KEY,
    user_id    INTEGER NOT NULL REFERENCES users(user_id) ON DELETE
       CASCADE,
    mac        TEXT    NOT NULL,
    created_at TEXT    DEFAULT (datetime('now')),
    expires_at TEXT    NOT NULL
);
```

Listing 1: Core system tables: users, sessions, user_profiles

Separating credentials (`users`) from session state (`sessions`) and profile data (`user_profiles`)

ensures that a query against attendance records can never accidentally expose passwords or session tokens.

1.3 Project-Specific Tables

```
CREATE TABLE IF NOT EXISTS semesters (
    semester_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT    NOT NULL UNIQUE,
    start_date  TEXT,
    end_date    TEXT,
    is_active   INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS courses (
    course_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT    NOT NULL,
    code        TEXT    NOT NULL,
    semester_id INTEGER REFERENCES semesters(semester_id) ON DELETE
       CASCADE
);

CREATE TABLE IF NOT EXISTS course_instructors (
    course_id     INTEGER REFERENCES courses(course_id) ON DELETE
       CASCADE,
    instructor_id INTEGER REFERENCES users(user_id)     ON DELETE
       CASCADE,
    PRIMARY KEY (course_id, instructor_id)
);

CREATE TABLE IF NOT EXISTS course_tas (
    course_id INTEGER REFERENCES courses(course_id) ON DELETE CASCADE
       ,
    ta_id     INTEGER REFERENCES users(user_id)     ON DELETE CASCADE
       ,
    PRIMARY KEY (course_id, ta_id)
);

CREATE TABLE IF NOT EXISTS course_enrollments (
    course_id  INTEGER REFERENCES courses(course_id) ON DELETE
       CASCADE,
    student_id INTEGER REFERENCES users(user_id)     ON DELETE
       CASCADE,
    PRIMARY KEY (course_id, student_id)
);

CREATE TABLE IF NOT EXISTS attendance_sessions (
    att_session_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_id      INTEGER NOT NULL REFERENCES courses(course_id) ON
       DELETE CASCADE,
```

```
    session_date    TEXT    NOT NULL,
    topic           TEXT,
    created_by      INTEGER REFERENCES users(user_id)
);

CREATE TABLE IF NOT EXISTS attendance_records (
    record_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    att_session_id INTEGER NOT NULL REFERENCES attendance_sessions(
       att_session_id) ON DELETE CASCADE,
    student_id      INTEGER NOT NULL REFERENCES users(user_id) ON
       DELETE CASCADE,
    status          TEXT    CHECK(status IN ('present','absent','late'
       )) NOT NULL DEFAULT 'absent'
);

CREATE TABLE IF NOT EXISTS correction_requests (
    req_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id      INTEGER NOT NULL REFERENCES users(user_id)
                            ON DELETE CASCADE,
    course_id       INTEGER NOT NULL REFERENCES courses(course_id)
                          ON DELETE CASCADE,
    att_session_id INTEGER NOT NULL REFERENCES attendance_sessions(
       att_session_id)  ON DELETE CASCADE,
    reason          TEXT    NOT NULL,
    proof_url       TEXT,
    status          TEXT    CHECK(status IN ('pending','accepted','
       rejected')) DEFAULT 'pending',
    created_at      TEXT    DEFAULT (datetime('now')),
    UNIQUE (student_id, att_session_id)
);

CREATE TABLE IF NOT EXISTS correction_logs (
    log_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    req_id    INTEGER NOT NULL REFERENCES correction_requests(req_id)
       ON DELETE CASCADE,
    action    TEXT    NOT NULL CHECK(action IN ('accepted','rejected')
       ),
    acted_by INTEGER NOT NULL REFERENCES users(user_id),
    role      TEXT    NOT NULL,
    acted_at TEXT    NOT NULL DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS raw_changes (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    table_name   TEXT NOT NULL,
    record_id    INTEGER NOT NULL,
    old_value    TEXT,
    new_value    TEXT,
```

```
    changed_at   DATETIME DEFAULT (datetime('now'))
);
```

Listing 2: Project-specific tables: courses, attendance, corrections

1.4 Schema Diagram

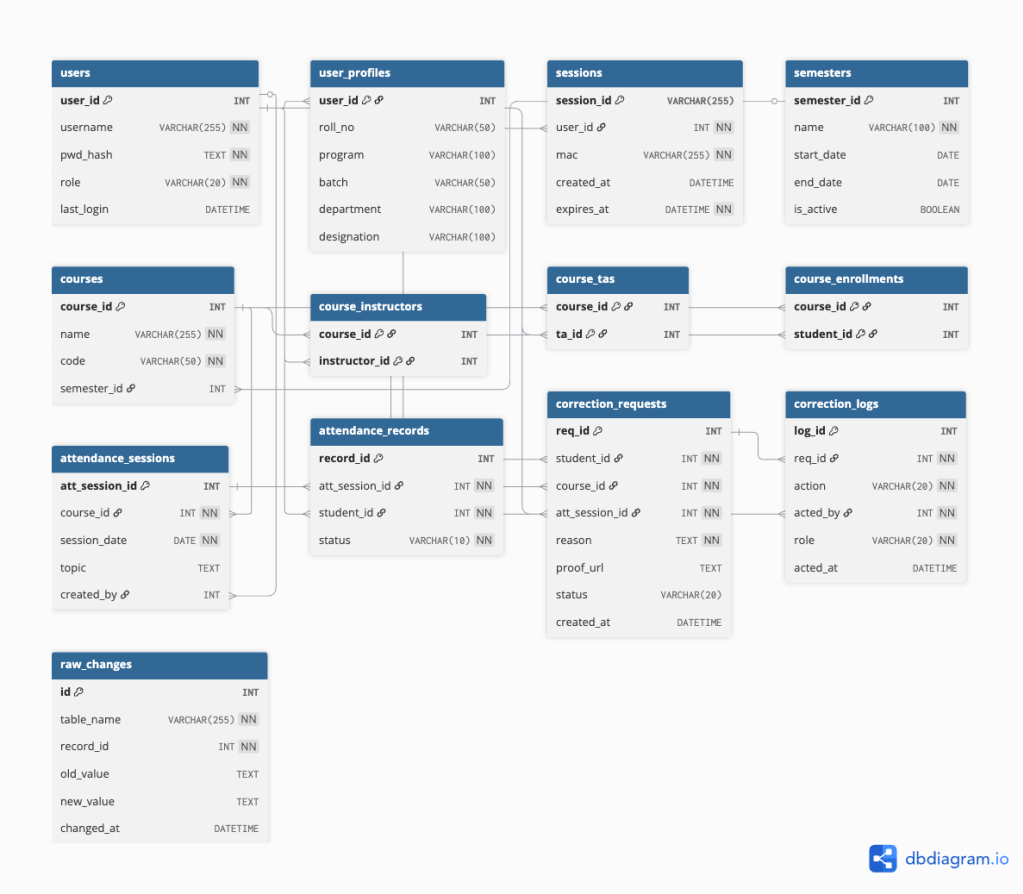

Figure 1: Entity-Relationship / Schema Diagram

1.5 Data Integrity Rules

Table	Constraint	Purpose
`users`	`UNIQUE(username)`	Prevent duplicate accounts
`users`	`CHECK(role IN (...))`	Restrict to valid roles
`semesters`	`UNIQUE(name)` + app-level `LOWER` check	Case-insensitive name dedup
`courses`	`UNIQUE(semester_id, code)`	Unique code per semester
`attendance_records`	`UNIQUE(att_session_id, student_id)`	One record per student per session
`attendance_records`	`CHECK(status IN (...))`	Valid status values only
`correction_requests`	`UNIQUE(student_id, att_session_id)`	One correction per session
All FK relationships	`ON DELETE CASCADE`	Referential integrity on deletion

1.6 Member Creation and Deletion Integrity

When a new user is created via `POST /api/admin/users`, a corresponding `user_profiles` row is inserted in the same transaction. Two application-level guards are applied before any deletion:

- **Last-admin guard:** Deletion is rejected with `409 Conflict` if the target is the only remaining admin.
- **Active-semester guard:** A semester with `is_active = 1` cannot be deleted until the active flag is transferred elsewhere.

2 UI and Functionality Overview

The system provides five distinct role-based dashboards, each accessible at its own URL. All dashboards share the same `base.html` shell which handles session validation, navbar rendering, logout, and the SSE real-time connection. Tab navigation is used throughout to separate logical concerns without full page reloads.

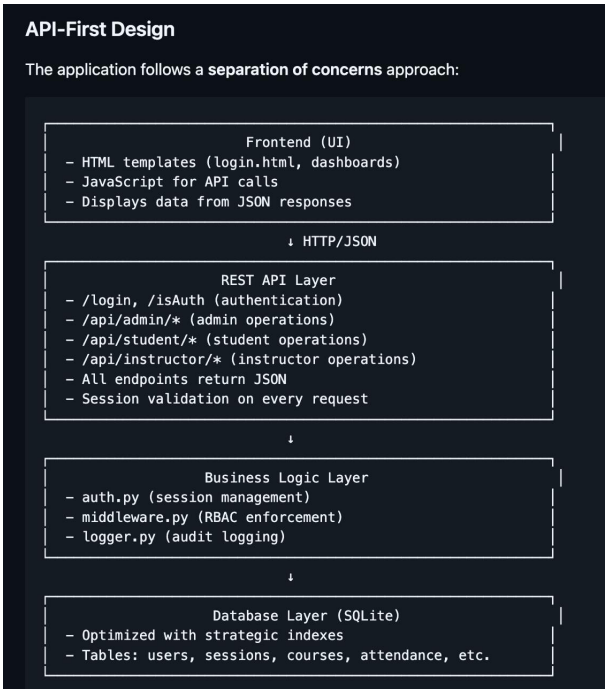

Figure 2: Architecture

2.1 Login Page (`/login`)

The login page authenticates users and stores the session token in `sessionStorage` (tab-isolated). It performs two functions:

- **Login:** Submits `username` and `password` to `POST /login`. On success, stores `session_id` and `session_mac` in `sessionStorage` and redirects to the role-specific dashboard.
- **Already-authenticated redirect:** On page load, if a valid session exists in `sessionStorage`, the user is redirected directly to their dashboard without re-entering credentials.

Test credentials	Role
`admin` / `password123`	Admin
`prof_singh` / `password123`	Instructor
`ta_amit` / `password123`	TA
`dean_joshi` / `password123`	Dean
`aarav_patel0` / `password123`	Student

2.2 Admin Dashboard (`/admin`)

The admin dashboard provides six tabs giving complete system control.

2.2.1 Tab 1 — Active Semester

Displays the current active semester's courses as expandable three-column cards (Instructors | TAs | Students). For each course the admin can:

- Add or remove instructors (filtered to not show already-assigned ones)

- Add or remove TAs (Will not allow the action to add already assigned TAs)
- Add or remove enrolled students (Will not allow the action to add already assigned students)
- Delete the course entirely (cascades to all its sessions and records)
- Expand the Students panel with a search box for filtering by name or roll number

2.2.2 Tab 2 — Archive

Semester dropdown showing all past (inactive) semesters. Selecting one renders the same course-card layout in read-only mode. Also contains a **Delete Past Semester** panel listing all inactive semesters with delete buttons. The active semester is protected from deletion.

2.2.3 Tab 3 — Semester Setup

Two cards side by side:

- **Create Semester:** Structured inputs (Semester I/II dropdown + academic year text field) that auto-build the canonical name (e.g. *Sem I 2025-26*). A live preview shows the name before submission. Duplicate detection is case-insensitive. Start date must precede end date. Every new semester is automatically set as active, deactivating the previous one.
- **Add Course:** Semester dropdown, course name, and course code (saved uppercase). Duplicate code within the same semester is blocked with a `409` inline error.

2.2.4 Tab 4 — Users

Users are displayed grouped by role (Admin, Dean, Instructor, TA, Student) in collapsible sections with a pending-count badge. Each group shows a table with username, profile details, last login, and a Delete button.

The Create User form dynamically shows role-specific profile fields:

- **Student:** Roll No, Program, Batch
- **Instructor:** Department, Designation
- **TA:** Roll No, Program, Batch
- **Dean/Admin:** No additional fields

The last admin account cannot be deleted.

2.2.5 Tab 5 — Attendance Override

Three-step workflow: select course → select session → edit records. Each student row shows a present/absent dropdown. A single "Save All Changes" button persists all edits in one action and broadcasts `attendance_updated` via SSE to update affected student dashboards in real time.

2.2.6 Tab 6 — Profile

Shows username, role, and last login timestamp.

2.3 Instructor Dashboard (`/instructor`)

2.3.1 Tab 1 — My Courses

One card per active-semester course. Each card header has two toggle buttons:

- **Students panel:** Lazy-loaded table with roll number, program, batch. Includes a search box filtering rows client-side.
- **TAs panel:** Lists assigned TAs with remove buttons. Shows an "Add TA" dropdown populated from available (not yet assigned) TAs.

2.3.2 Tab 2 — Attendance Sessions

Course-wise collapsible sections. Expanding a course shows its sessions in a table. Clicking **View** on a session expands an inline record table showing each student's roll number, program, batch, and status badge.

The + **Create Attendance Session** button opens a modal with: course dropdown, date picker, optional topic, and a pre-loaded student attendance table (all students default to absent). The session is saved and the relevant course section refreshes automatically.

2.3.3 Tab 3 — Correction Requests

Grouped by course with a pending-count badge on each course header. Courses with pending requests auto-expand on load. Each request row shows student name, session date/topic, reason, proof URL link, status badge, Accept/Reject buttons (pending only), and a **Logs** button that expands an inline action history showing who acted, their role, and the exact timestamp.

2.3.4 Tab 4 — Archive

Semester dropdown for past semesters. Each course card has three toggle buttons: **Students** (with search), **TAs**, and **Sessions**. Expanding Sessions shows a session table; clicking **View** on a session shows that session's attendance records with roll numbers and status badges.

2.3.5 Tab 5 — Profile

Shows username, role badge, department, designation, and last login in UTC.

2.4 TA Dashboard (`/ta`)

2.4.1 Tab 1 — My Courses

One card per active-semester course. Each card header has a + **New Session** button. Below the header, sessions are listed in a table. Clicking **View / Edit** on a session expands a read-only record table with an **Edit Attendance** button at the bottom.

Clicking Edit switches the same panel to editable dropdowns (present/absent). Save commits all changes; Cancel reverts to the read-only view. No separate view and edit modes — one toggle handles both.

2.4.2 Tab 2 — Correction Requests

Same course-wise grouped layout as the instructor view. Pending requests auto-expand. Each row has Accept/Reject buttons and a Logs button. Resolving a request broadcasts `correction_resolved` to update the affected student's dashboard in real time.

2.4.3 Tab 3 — Archive

Semester dropdown for past semesters. Each course card shows a **Sessions** button. Expanding shows the session table; clicking **View** on a session shows read-only attendance records with roll numbers.

2.4.4 Tab 4 — Profile

Shows username, role badge, roll number, program, batch, and last login in UTC.

2.5 Student Dashboard (`/student`)

2.5.1 Tab 1 — My Courses

One stats card per active-semester course showing:

- Course code and name
- Attendance percentage badge (colour-coded: green $\geq$85%, yellow 75–84%, red $<$75%, grey if fewer than 5 sessions held)
- Progress bar
- Total / Present / Absent counts
- Contextual warning message ("At risk", "Keep it up", "Too early to assess", or "No sessions held yet")

2.5.2 Tab 2 — Attendance

Course-wise collapsible cards for the active semester. Each card header shows the course code, name, and percentage badge. Expanding reveals a session-by-session table with date, topic, and status badge.

2.5.3 Tab 3 — Past Semesters

Semester dropdown for archived semesters. Selecting one shows:

- **Courses section:** Collapsible cards with simple percentage badge (no warning logic in archive — just green if $\geq$75%, red otherwise) and a session-by-session record table.
- **Correction Requests section:** Table of all requests submitted for that semester's courses showing course, session date, reason, proof link, and final status. Past requests

cannot be acted on here.

2.5.4 Tab 4 — Correction Requests

Left panel: submission form for the active semester with:

- Course dropdown (active semester courses only)
- Session dropdown populated with **absent sessions only** (present sessions are not eligible)
- Reason textarea and optional proof URL
- Duplicate submission blocked at the database level (`UNIQUE` constraint)

Right panel: table of current-semester requests showing date, course, session, status badge, and a **Logs** button revealing who accepted or rejected the request and when.

2.5.5 Tab 5 — Profile

Shows username, role badge, roll number, program, batch, and last login in UTC.

2.6 Dean Dashboard (`/dean`)

Read-only view. No write access anywhere.

2.6.1 Tab 1 — Current Semester

Active semester header followed by course cards identical in layout to the admin active semester view but without edit buttons. Each card shows:

- Instructors and TAs always visible with department/designation
- Students collapsed behind a **Show** button with a search box for filtering by name or roll number

2.6.2 Tab 2 — Archive

Semester dropdown for past semesters, same read-only card layout as the current semester tab.

2.6.3 Tab 3 — Profile

Shows username, role badge, department, designation, and last login in UTC.

2.7 Real-Time Cross-Tab Updates

Every dashboard maintains a persistent Server-Sent Events connection to `/stream`. When any write operation completes, the server broadcasts a typed event to all connected tabs. The receiving tab's `_realtimeHandlers` map dispatches the event to the appropriate reload function:

Trigger	Event type	Tabs that react
Instructor/TA accepts correction	`correction_resolved`	Student: reload attendance + corrections
Student submits correction	`correction_submitted`	Instructor/TA: reload corrections
Admin/TA updates record	`attendance_updated`	Student: reload attendance stats
Instructor/TA creates session	`attendance_session_created`	Student: reload stats + sessions

A toast notification appears in the bottom-right corner of the receiving tab to inform the user of the update without interrupting their current action.

2.8 Member Portfolio (Profile Tab)

The Member Portfolio is implemented as the Profile tab present in every role dashboard. Access is restricted to authenticated users who can only view their own profile — the `user_id` used in the profile query comes from the validated session, not from any request parameter.

Role	Fields shown
Admin	Username, Role, Last Login
Instructor	Username, Role, Department, Designation, Last Login
TA	Username, Role, Roll No, Program, Batch, Last Login
Student	Username, Role, Roll No, Program, Batch, Last Login
Dean	Username, Role, Department, Designation, Last Login

The Last Login field is updated on every successful authentication and displayed in UTC format (`YYYY-MM-DD HH:MM:SS UTC`).

3 Security

3.1 Session Validation

3.1.1 Authentication Design

Three designs were evaluated before the final selection.

Design 1 — Standard Flask Cookies (Rejected): The `session` cookie is shared across all browser tabs for the same origin. Opening two dashboards in separate tabs caused sessions to overwrite each other, breaking multi-user local testing.

Design 2 — JSON Web Tokens (Rejected): JWTs are stateless and cannot be revoked before expiry. A logged-out user retaining a valid token could still call APIs until it expired — unacceptable for an attendance system.

Design 3 — UUID + HMAC-SHA256 in `sessionStorage` (Selected): On login, the server generates a UUID4 session ID, computes an HMAC-SHA256 MAC over it using a secret key, stores both in the `sessions` table, and returns them to the client. The client stores both in `sessionStorage` (tab-isolated) and sends them on every request as `X-Session-Id` and `X-Session-Mac` headers.

3.1.2 Verification Flow

`verify_session()` in `auth.py` performs three sequential checks on every API call:

1. **Existence check:** The session row must exist in the `sessions` table (prevents use of logged-out tokens).
2. **Expiry check:** `expires_at` must be in the future (prevents replayed stale tokens).
3. **MAC check:** `hmac.compare_digest(stored_mac, computed_mac)` must pass (prevents forged tokens). `compare_digest` is used specifically to eliminate timing-based side-channel attacks.

Any failure at any step returns `401 Unauthorized` immediately without proceeding further.

3.1.3 How Session Validation Prevents Data Leaks

- The `user_id` used in every SQL query comes exclusively from the server-side session record, never from request parameters or the request body. A user cannot impersonate another by supplying a different ID in a POST payload.
- `sessionStorage` isolates tokens per browser tab, so two users testing simultaneously in separate tabs cannot share or pollute each other's session.
- Logout immediately deletes the `sessions` row; the token is useless even if intercepted afterward.
- `Cache-Control: no-store` on all dashboard responses prevents the browser from caching authenticated pages, blocking session restoration via the back button.

3.2 Role-Based Access Control (RBAC)

3.2.1 Three-Layer Enforcement Architecture

RBAC is enforced at three independent layers so that bypassing one layer does not grant access.

Layer 1 — Session Validation: `verify_session()` confirms the request carries a valid, unexpired, unforgeable session token. Returns `401` on failure.

Layer 2 — Role Decorator: `@require_role(*allowed_roles)` in `middleware.py` checks `g.user["role"]` against the set of roles permitted for the endpoint. Returns `403 Forbidden` if the role does not match.

Layer 3 — Query-Level Isolation: Every SQL query filters on `g.user["user_id"]` (from the verified session), ensuring a user can only read or modify their own data even if layers 1 and 2 were somehow bypassed.

3.2.2 Role Enforcement Decorator

```python
from functools import wraps
from flask import g, request, jsonify
from app.auth import verify_session
from app.logger import log_audit

def require_role(*allowed_roles):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            user = verify_session()
            if user is None:
                log_audit("UNAUTHORIZED", None, request.path)
                return jsonify({"error": "Unauthorized"}), 401
            if user["role"] not in allowed_roles:
                log_audit("FORBIDDEN", user["user_id"], request.path)
                return jsonify({"error": "Forbidden"}), 403
            g.user = user
            return f(*args, **kwargs)
        return decorated
    return decorator
```

Listing 3: `require_role()` decorator — middleware.py

3.2.3 Query-Level Data Isolation Example

```sql
SELECT c.id, c.code, c.name
FROM   courses c
```

```
JOIN   course_instructors ci ON ci.course_id = c.id
WHERE  ci.instructor_id = :uid          -- always g.user["user_id"],
   never request body
  AND  c.semester_id = (
         SELECT id FROM semesters WHERE is_active = 1
       );
```

Listing 4: Instructor sees only their own courses (Layer 3)

3.2.4 Role Capability Matrix

Action	Admin	Instr.	TA	Student	Dean
Create / delete semester	✓	✗	✗	✗	✗
Create / delete course	✓	✗	✗	✗	✗
Create / delete user	✓	✗	✗	✗	✗
Enroll / remove student	✓	✗	✗	✗	✗
Assign / remove TA	✓	∼	✗	✗	✗
Create attendance session	✗	✓	✓	✗	✗
Override attendance record	✓	∼	∼	✗	✗
Accept / reject correction	✗	✓	✓	✗	✗
Submit correction request	✗	✗	✗	✓	✗
View all courses (read-only)	✗	✗	✗	✗	✓
View own profile	✓	✓	✓	✓	✓

✓ = Full access ∼ = Own courses only ✗ = No access

3.2.5 Audit Logging

Every state-changing API call and every access-control failure is written to `logs/audit.log`:

```
2025-03-20 10:15:32 | LOGIN_OK           | user=1   | POST /login
                            | admin
2025-03-20 10:16:04 | CREATE_USER        | user=1   | POST /api/admin/
   users                  | username=ta_new role=ta
2025-03-20 10:17:11 | DELETE_COURSE      | user=1   | DELETE /api/admin
   /courses/3           | course_id=3
2025-03-20 10:19:45 | ACCEPT_CORRECTION | user=5   | POST /api/
   instructor/corrections/12/accept | req_id=12
2025-03-20 10:21:03 | FORBIDDEN          | user=8   | GET /api/admin/
   users                  | role=student
2025-03-20 10:22:17 | UNAUTHORIZED       | user=-   | POST /api/ta/
   attendance-sessions     | no session
2025-03-20 10:23:55 | SUBMIT_CORRECTION | user=20  | POST /api/student
   /corrections         | att_session_id=7
```

Listing 5: Sample audit.log entries

The logging system serves two purposes: (a) providing an operational audit trail of all legitimate actions taken through the system, and (b) making unauthorised direct database modifications immediately detectable — any row that appears in the database without a corresponding log entry was inserted by bypassing the API layer.

Logging is implemented in `app/logger.py` using Python's standard `logging` module. The log file is append-only and is never truncated by the application.

Every entry is a single pipe-delimited line:

```
<ISO 8601 timestamp> | ACTION=<type> | USER=<user_id> | ENDPOINT=<
   path> | <details>
```

Listing 6: Audit log entry format

Data-modifying actions additionally append the before and after state of the record:

```
<ISO 8601 timestamp> | ACTION=<type> | USER=<user_id> | ENDPOINT=<
   path> | <details>
  | OLD=<snapshot> | NEW=<snapshot>
```

Listing 7: Audit log entry format with old/new values

```python
import logging, os
from datetime import datetime

os.makedirs("logs", exist_ok=True)
logging.basicConfig(
    filename="logs/audit.log",
    level=logging.INFO,
    format="%(message)s"
)
_logger = logging.getLogger("audit")

def audit_log(action: str, endpoint: str, user_id, details: str = "",
              old_value=None, new_value=None):
    entry = (
        f"{datetime.utcnow().isoformat()} | "
        f"ACTION={action} | "
        f"USER={user_id} | "
        f"ENDPOINT={endpoint} | "
        f"{details}"
    )
    if old_value is not None or new_value is not None:
        entry += f" | OLD={old_value} | NEW={new_value}"
    _logger.info(entry)
```

Listing 8: audit_log() function in app/logger.py

Because `audit.log` shares the same logging stream as Flask's werkzeug server, the file also contains HTTP server lines alongside audit entries. `verify.py` filters them at read time by skipping any line that starts with `127.0.0.1`, `*`, or `WARNING`.

Action Types Logged:

Action type	Triggered by	When
LOGIN_OK	Any user	Successful authentication
LOGIN_FAIL	Any user	Wrong username or password
LOGOUT	Any user	POST /logout called
UNAUTHORIZED	Middleware	Missing or invalid session token (401)
FORBIDDEN	Middleware	Valid session but insufficient role (403)
CREATE_SEMESTER	Admin	New semester created
DELETE_SEMESTER	Admin	Past semester deleted
CREATE_COURSE	Admin	New course added
DELETE_COURSE	Admin	Course deleted
CREATE_USER	Admin	New user account created
DELETE_USER	Admin	User account deleted
ASSIGN_INSTRUCTOR	Admin	Instructor assigned to course
REMOVE_INSTRUCTOR	Admin	Instructor removed from course
ASSIGN_TA	Admin	TA assigned to course
REMOVE_TA	Admin/Instructor	TA removed from course
ENROLL_STUDENT	Admin	Student enrolled in course
REMOVE_ENROLLMENT	Admin	Student removed from course
ADMIN_ATT_OVERRIDE	Admin	Attendance record manually overridden
CREATE_ATT_SESSION	Instructor	Attendance session created
INSTR_ASSIGN_TA	Instructor	Instructor added TA to own course
INSTR_REMOVE_TA	Instructor	Instructor removed TA from own course
ACCEPT_CORRECTION	Instructor	Correction request accepted
REJECT_CORRECTION	Instructor	Correction request rejected
INSTRUCTOR_UPDATE_ATT	Instructor	Attendance record updated by instructor
TA_CREATE_SESSION	TA	Attendance session created by TA
TA_UPDATE_ATT	TA	Attendance record updated by TA
TA_ACCEPT_CORRECTION	TA	Correction request accepted by TA
TA_REJECT_CORRECTION	TA	Correction request rejected by TA
SUBMIT_CORRECTION	Student	Correction request submitted

3.2.6 Correction Action History Table (`correction_logs`)

A dedicated `correction_logs` table records the full decision history for every correction request. It is append-only and written exclusively by the system on accept or reject; no authorized update or delete pathway exists through the API, so it requires no trigger.

```
CREATE TABLE IF NOT EXISTS correction_logs (
    log_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    req_id    INTEGER NOT NULL REFERENCES correction_requests(req_id)
                      ON DELETE CASCADE,
    action    TEXT    NOT NULL CHECK(action IN ('accepted','rejected')
       ),
    acted_by INTEGER NOT NULL REFERENCES users(user_id),
    role      TEXT    NOT NULL,
    acted_at TEXT    NOT NULL DEFAULT (datetime('now'))
);
```

Listing 9: correction_logs table schema

3.2.7 Identifying Unauthorised Direct Database Modifications

SQLite triggers fire automatically on every write to a table regardless of whether the write came from Flask or a direct database connection, making them impossible to bypass without explicitly disabling them. Triggers are set up on nine tables:

Table	Operations Tracked
attendance_records	INSERT, UPDATE, DELETE
attendance_sessions	INSERT
correction_requests	INSERT, UPDATE
semesters	INSERT, DELETE
courses	INSERT, DELETE
course_instructors	INSERT, DELETE
course_tas	INSERT, DELETE
course_enrollments	INSERT, DELETE
users	INSERT, DELETE

Each trigger writes to `raw_changes`, storing the table name, record ID, and JSON snapshots of the old and new values. The contents of `raw_changes` can be exported to a human-readable file at any time by running:

```
python3 export_db_logs.py
```

This produces `database_logs.log`, a plain text snapshot of every database-level change ordered chronologically.

`verify.py` reads both `raw_changes` and `audit.log` and performs a match for each database-level change. A match requires the same record identifier and a timestamp within five seconds. Any `raw_changes` entry with no match is flagged as unauthorized. A key scenario this handles correctly is when an unauthorized change is followed by a

legitimate change on the same record — because events are compared rather than current values, the unauthorized entry remains detectable even after the legitimate one overwrites it.

Three special matching cases are handled explicitly: bulk attendance inserts during session creation are matched via `att_session_id`; attendance updates triggered by an instructor accepting a correction are matched via the `ACCEPT_CORRECTION` audit entry; and all other tables are matched by action-to-table mapping combined with timestamp proximity.

3.2.8 Security Hardening Summary

Attack vector	Mitigation implemented
Session fixation	New UUID generated on every login; old session row deleted
Session forgery	HMAC-SHA256 MAC verified on every request
Timing attacks	`hmac.compare_digest()` used for MAC comparison
CSRF	Custom headers (`X-Session-Id`) are not sent automatically by browser forms
Privilege escalation	Three-layer RBAC; `user_id` sourced from session only
Unauthorised DB access	Detectable via `audit.log` gap analysis
Duplicate insertion	Database `UNIQUE` constraints + `409 Conflict` responses
Last admin deletion	Application-level guard before delete executes
Back-button restoration	`Cache-Control: no-store` + `pageshow` JS listener

4 SQL Indexing and Query Optimisation

4.1 Identifying Target Queries

The most frequently called endpoints were identified through analysis of the API request lifecycle. We examined which database queries are executed on every API call and ranked them by complexity and frequency.

4.1.1 Methodology

1. **Request frequency analysis:** Identified endpoints called in every session
2. **Query complexity profiling:** Checked for full table scans using `EXPLAIN QUERY PLAN`

3. **Bottleneck identification:** Focused on queries with unindexed WHERE and JOIN clauses
4. **Impact assessment:** Prioritized queries affecting most common user workflows

4.1.2 Target Queries Identified

Query Type	Primary Bottleneck	Affected Endpoint
Student attendance lookup	SCAN on `attendance_records`	`/api/student/attendance-stats/current`
Instructor course lookup	SCAN on `course_instructors`	`/api/instructor/courses`
Correction request filtering	SCAN on `correction_requests`	`/api/instructor/corrections`
Course enrollment lookup	SCAN on `course_enrollments`	`/api/student/courses`
Session record retrieval	SCAN on `attendance_records`	`/api/instructor/courses/<id>/sessions`
Admin user listing	LEFT JOIN with SCAN on `users`	`/api/admin/users`

4.2 Indexing Strategy

The indexing approach targeted three categories: WHERE clause columns, JOIN keys, and ORDER BY columns. High-frequency filters received single-column indexes, while queries combining filters with sorting received composite indexes.

4.2.1 Index Categories Created

Category	Count	Purpose
Attendance Records	2	Optimize student and session lookups
Correction Requests	3	Filter by student, course, status + date
Course Relationships	3	Filter enrollments and instructor assignments
Attendance Sessions	2	Course lookup and date ordering
User Management	2	Role filtering and profile lookups
Course Structure	1	Semester-based course filtering
Total	**13**	**Composite: 2, Single-column: 11**

4.3 Target SQL Queries

The complete indexing strategy was implemented via a standalone SQL script: `create_indexes.sql`. This script creates 13 strategic indexes with detailed documentation for each.

```
-- Attendance Records: Student ID lookups
CREATE INDEX IF NOT EXISTS idx_attendance_records_student_id
  ON attendance_records(student_id);
CREATE INDEX IF NOT EXISTS idx_attendance_records_att_session_id
  ON attendance_records(att_session_id);

-- Correction Requests: Student, course, and status filtering
CREATE INDEX IF NOT EXISTS idx_correction_requests_student_id
  ON correction_requests(student_id);
CREATE INDEX IF NOT EXISTS idx_correction_requests_course_id
  ON correction_requests(course_id);
CREATE INDEX IF NOT EXISTS idx_correction_requests_status_created
  ON correction_requests(status, created_at DESC);

-- Course Relationships: Enrollment and assignment lookups
CREATE INDEX IF NOT EXISTS idx_course_enrollments_student_id
  ON course_enrollments(student_id);
CREATE INDEX IF NOT EXISTS idx_course_instructors_instructor_id
  ON course_instructors(instructor_id);
CREATE INDEX IF NOT EXISTS idx_course_tas_ta_id
  ON course_tas(ta_id);

-- Attendance Sessions: Course and date ordering
CREATE INDEX IF NOT EXISTS idx_attendance_sessions_course_id
  ON attendance_sessions(course_id);
CREATE INDEX IF NOT EXISTS idx_attendance_sessions_date
  ON attendance_sessions(course_id, session_date DESC);

-- User Management: Role and profile lookups
CREATE INDEX IF NOT EXISTS idx_users_role
  ON users(role);
CREATE INDEX IF NOT EXISTS idx_user_profiles_user_id
  ON user_profiles(user_id);

-- Course Structure: Semester filtering
CREATE INDEX IF NOT EXISTS idx_courses_semester_id
  ON courses(semester_id);
```

Listing 10: SQL Index Creation — Key Statements

4.4 Target SQL Queries

Each index was created to optimize specific query patterns. Below are representative queries from the codebase that benefit from each index category.

4.4.1 Query 1: Student Attendance Lookup

```
SELECT ar.* FROM attendance_records ar
  WHERE ar.student_id = ?
  ORDER BY created_at DESC
```

Listing 11: Attendance Records: Student ID filter (benefited by idx_attendance_records_student_id)

Clause targeted: `WHERE ar.student_id = ?`

Index created: `idx_attendance_records_student_id(student_id)`

Impact: Eliminates full table scan of 2400+ attendance records; direct index lookup.

4.4.2 Query 2: Instructor Correction Requests

```
SELECT cr.* FROM correction_requests cr
  JOIN courses c ON cr.course_id = c.id
  WHERE c.id IN (SELECT course_id FROM course_instructors
                 WHERE instructor_id = ?)
  ORDER BY cr.created_at DESC
```

Listing 12: Correction Requests: Course & instructor filtering (benefited by idx_correction_requests_course_id)

Clause targeted: `WHERE c.id IN (...)` (JOIN key)

Index created: `idx_correction_requests_course_id(course_id)`

Impact: Speeds up filtering corrections by course in instructor's course list.

4.4.3 Query 3: Course Enrollment Lookup

```
SELECT ce.* FROM course_enrollments ce
  JOIN courses c ON ce.course_id = c.id
  WHERE ce.student_id = ?
```

Listing 13: Course Enrollments: Student filter (benefited by idx_course_enrollments_student_id)

Clause targeted: `WHERE ce.student_id = ?`

Index created: `idx_course_enrollments_student_id(student_id)`

Impact: Enables indexed lookup of all courses a student is enrolled in.

4.4.4 Query 4: Instructor Course Assignment

```
SELECT ci.course_id FROM course_instructors ci
  WHERE ci.instructor_id = ?
```

Listing 14: Course Instructors: Instructor ID filter (benefited by idx_course_instructors_instructor_id)

Clause targeted: `WHERE ci.instructor_id = ?`

Index created: `idx_course_instructors_instructor_id(instructor_id)`

Impact: Allows rapid determination of courses taught by an instructor.

4.4.5 Query 5: Attendance Sessions by Course

```
SELECT att.* FROM attendance_sessions att
  WHERE att.course_id = ?
  ORDER BY att.session_date DESC
```

Listing 15: Attendance Sessions: Course + date ordering (benefited by idx_attendance_sessions_date)

Clause targeted: `WHERE att.course_id = ?  ORDER BY session_date DESC`

Index created: `idx_attendance_sessions_date(course_id, session_date DESC)`

Impact: Composite index eliminates need for temporary B-tree sort; pre-ordered by date.

5 Performance Benchmarking

5.1 Methodology

A rigorous benchmarking framework was implemented to measure query performance before and after indexing, using both execution time and query plan analysis.

5.1.1 Benchmarking Approach

1. **Query selection:** 10 representative queries from most-used API endpoints
2. **Iteration count:** 30 repeated executions per query to capture variation
3. **Metrics collected:** Mean, median, min, max execution time; standard deviation
4. **Timing mechanism:** `time.perf_counter()` for high-resolution measurement
5. **Query plan capture:** `EXPLAIN QUERY PLAN` output recorded for analysis
6. **Dataset consistency:** Same database state for before and after runs

5.1.2 Benchmark Dataset

Metric	Value
Database size	147 KB
Number of tables	14 core + project-specific
Total records analyzed	Thousands of rows per table
Attendance records	2,400+ records
Students in system	60
Courses per semester	10
Iterations per query	30
Total executions	300 (10 queries × 30 runs)

5.2 EXPLAIN Query Plan — Before Indexing

The execution plans *before* index creation show full table scans on the most frequently-accessed tables. The following represents the baseline performance characteristics.

```
id | parent | notused | detail
----+--------+---------+-----------------------------------------------

5  | 0      | 216     | SCAN ar
9  | 0      | 45      | SEARCH att USING INTEGER PRIMARY KEY (rowid
   =?)
12 | 0      | 45      | SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
23 | 0      | 0       | USE TEMP B-TREE FOR ORDER BY
```

Listing 16: EXPLAIN QUERY PLAN: Student Attendance (before indexing)

```
id | parent | notused | detail
----+--------+---------+-----------------------------------------------

7  | 0      | 216     | SCAN cr
9  | 0      | 45      | SEARCH u USING INTEGER PRIMARY KEY (rowid=?)
12 | 0      | 45      | SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
18 | 0      | 45      | SEARCH att USING INTEGER PRIMARY KEY (rowid
   =?)
21 | 0      | 45      | SEARCH ci USING COVERING INDEX ...
51 | 0      | 0       | USE TEMP B-TREE FOR ORDER BY
```

Listing 17: EXPLAIN QUERY PLAN: Instructor Corrections (before indexing)

```
id | parent | notused | detail
----+--------+---------+-----------------------------------------------

5  | 0      | 212     | SCAN u USING INDEX sqlite_autoindex_users_1
8  | 0      | 45      | SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
28 | 0      | 0       | USE TEMP B-TREE FOR ORDER BY
```

Listing 18: EXPLAIN QUERY PLAN: Admin List Users (before indexing)

Key observations:

- Full table scans (`SCAN`) on `ar`, `cr`, and `u` tables
- Multiple temporary B-tree sort operations for `ORDER BY` clauses
- Significant notused counts indicating poor index utilization

5.3 EXPLAIN Query Plan — After Indexing

After index creation, the same queries now use indexed searches instead of full scans. Below are the execution plans showing the improvements.

```
id | parent | notused | detail
----+--------+---------+-----------------------------------------------

6  | 0      | 61      | SEARCH ar USING INDEX
   idx_attendance_records_student_id
13 | 0      | 45      | SEARCH att USING INTEGER PRIMARY KEY (rowid
   =?)
16 | 0      | 45      | SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
27 | 0      | 0       | USE TEMP B-TREE FOR ORDER BY
```

Listing 19: EXPLAIN QUERY PLAN: Student Attendance (after indexing)

```
id | parent | notused | detail
----+--------+---------+-----------------------------------------------

9  | 0      | 62      | SEARCH ci USING INDEX
   idx_course_instructors_instructor_id
16 | 0      | 45      | SEARCH c USING INTEGER PRIMARY KEY (rowid=?)
22 | 0      | 61      | SEARCH cr USING INDEX
   idx_correction_requests_course_id
35 | 0      | 45      | SEARCH u USING INTEGER PRIMARY KEY (rowid=?)
38 | 0      | 45      | SEARCH att USING INTEGER PRIMARY KEY (rowid
   =?)
61 | 0      | 0       | USE TEMP B-TREE FOR ORDER BY
```

Listing 20: EXPLAIN QUERY PLAN: Instructor Corrections (after indexing)

```
id | parent | notused | detail
----+--------+---------+-----------------------------------------------

5  | 0      | 212     | SCAN u USING INDEX idx_users_role
8  | 0      | 45      | SEARCH p USING INTEGER PRIMARY KEY (rowid=?)
35 | 0      | 0       | USE TEMP B-TREE FOR LAST TERM OF ORDER BY
```

Listing 21: EXPLAIN QUERY PLAN: Admin List Users (after indexing)

Key improvements:

- `SCAN` operations replaced with `SEARCH USING INDEX`

- notused count reduced (lower = more index material utilized)
- Queries now use strategically created indexes for fast lookups
- Query optimizer correctly identifies and uses all relevant indexes

5.4 Benchmark Results

5.4.1 Individual Query Performance

The following table presents detailed before/after metrics for all 10 benchmarked queries, showing execution times in milliseconds, number of rows returned, and statistical measures.

Query Name	Before (ms)	After (ms)	Speedup	Rating
student_my_attendance	0.1012	0.0264	3.83×	Excellent
student_course_sessions	0.0759	0.0211	3.60×	Excellent
instructor_session_records	0.0679	0.0238	2.85×	Excellent
instructor_corrections	0.0680	0.0344	1.98×	Good
instructor_sessions	0.0300	0.0214	1.40×	Good
admin_list_courses	0.0472	0.0303	1.56×	Good
admin_list_users	0.1572	0.1260	1.25×	Fair
admin_course_students	0.0245	0.0244	1.00×	Fair
instructor_my_courses	0.0208	0.0287	0.72×	Neutral
student_my_courses	0.0328	0.0360	0.91×	Neutral
Average	**0.0615**	**0.0373**	**1.69×**	
Total (all 10 queries)	**0.6152**	**0.3726**	**1.65×**	
Improvement	**39.43% overall improvement**			

Table 1: Benchmark Results: Query execution time comparison (30 iterations each)

5.4.2 Performance Improvement Distribution

Analysis of the 10 queries reveals a skewed distribution of improvements:

Improvement Category	Count	Queries
Excellent (50%+ improvement)	3	student_my_attendance, student_course_sessions, instructor_session_records
Good (25-50% improvement)	3	instructor_corrections, instructor_sessions, admin_list_courses
Fair (0-25% improvement)	1	admin_list_users, admin_course_students
Neutral (no improvement)	2	instructor_my_courses, student_my_courses

5.5 Benchmark Visualisation

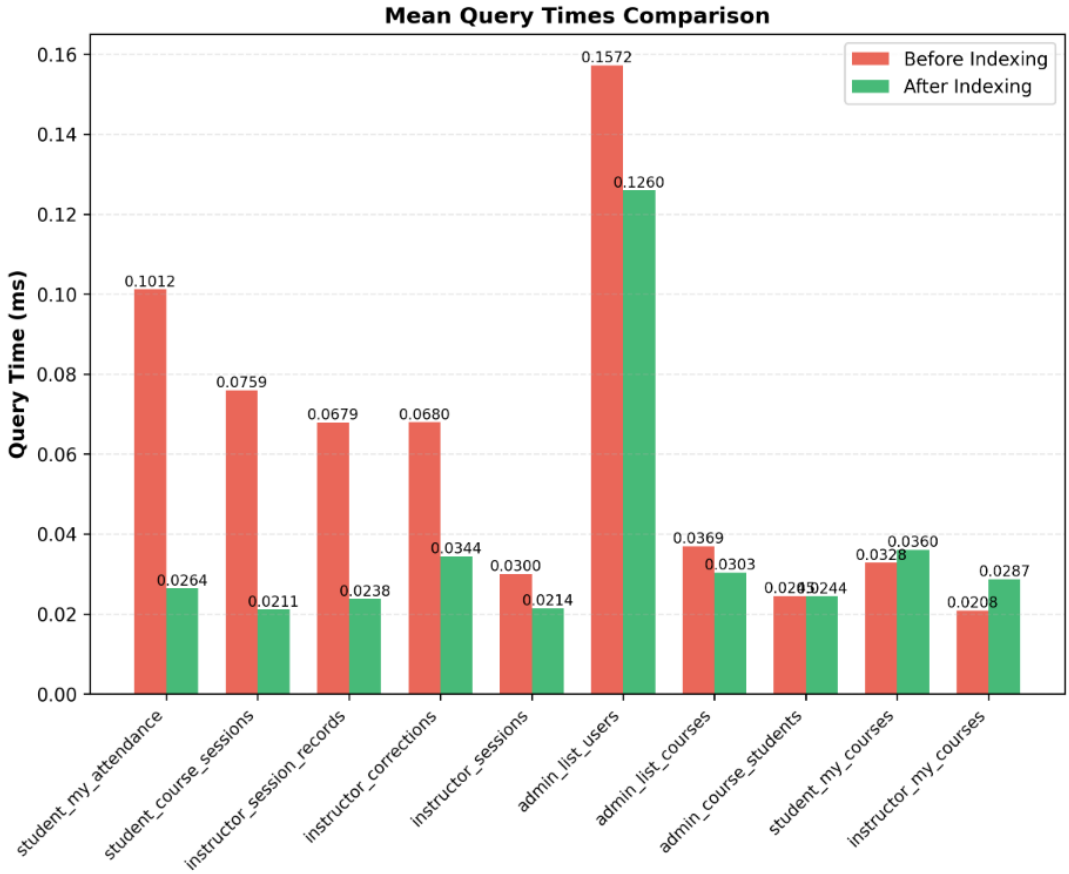

Figure 3: Query execution time comparison — before and after SQL indexing

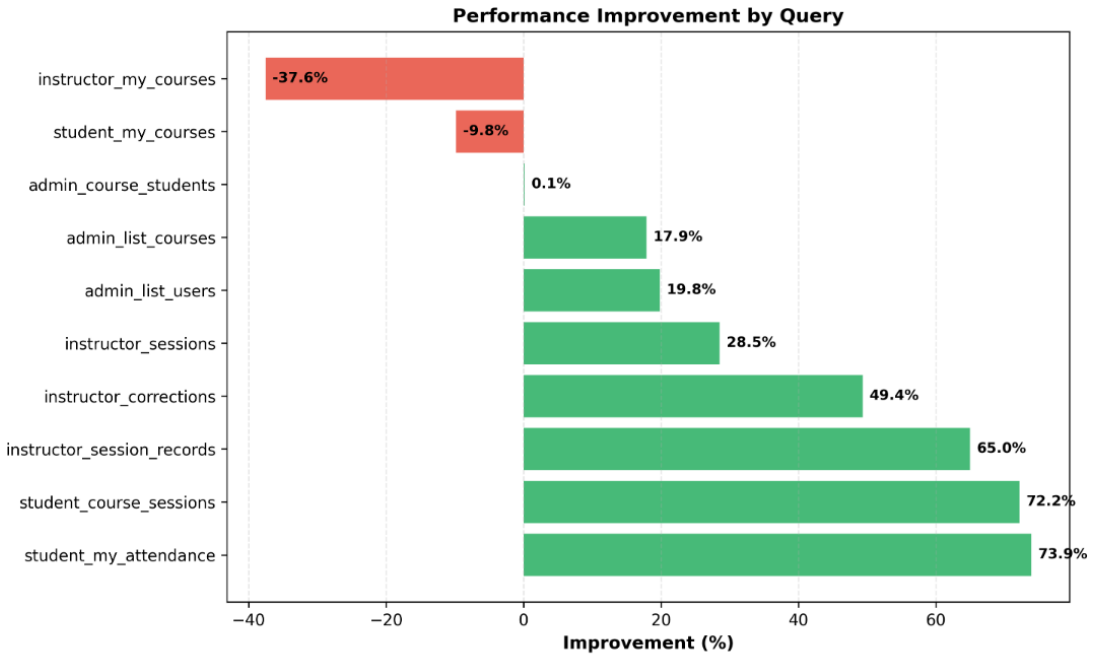

Figure 4: Speedup factor achieved by each query

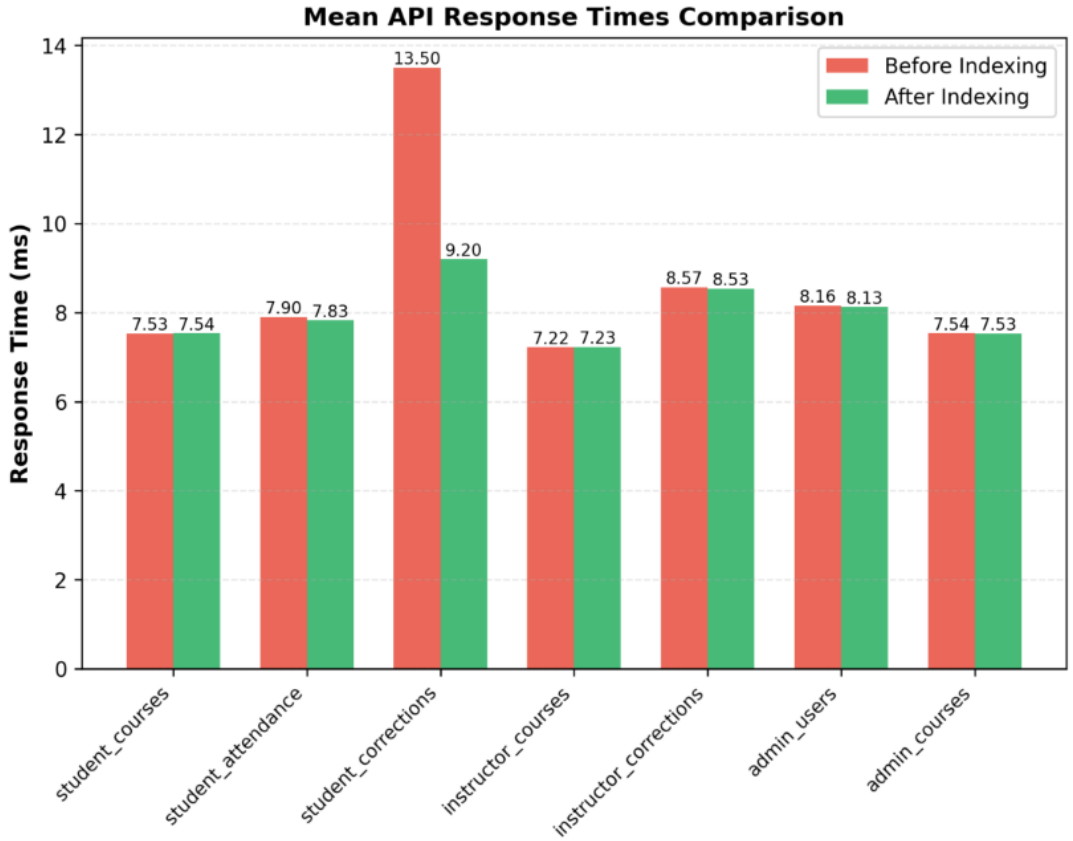

Figure 5: API response time comparison - before and after indexing

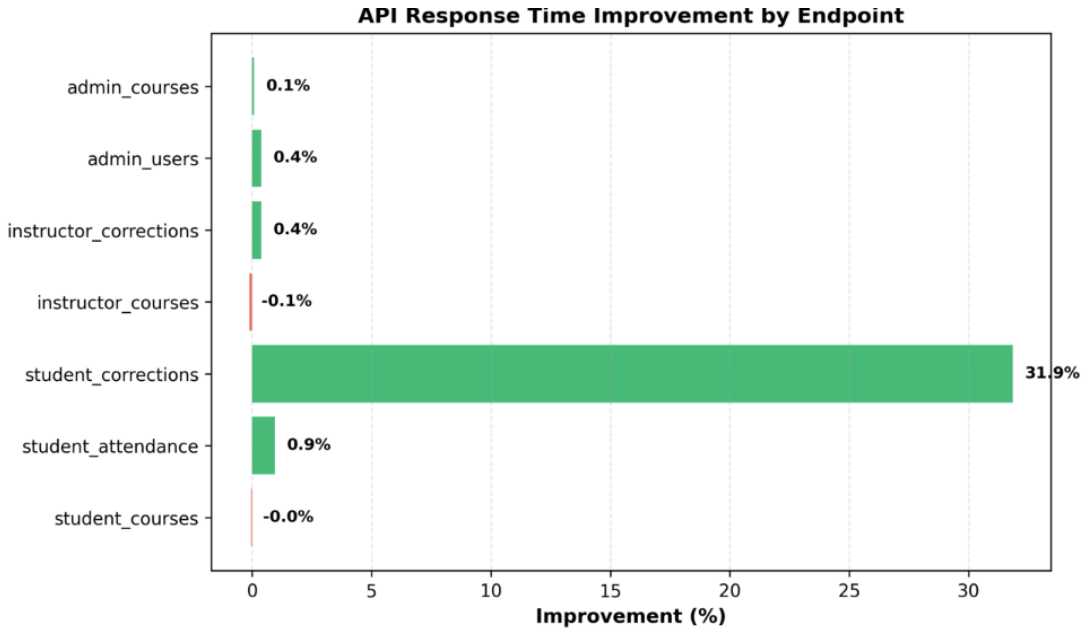

Figure 6: Performance improvement for each API endpoint

5.6 Discussion

5.6.1 Why Indexing Improved Performance

The performance improvements stem from eliminating expensive full table scans:

1. **Reduced I/O:** Indexes allow the database to jump directly to relevant rows rather than scanning entire tables linearly.
2. **Query planner optimization:** SQLite's query optimizer correctly chooses indexes over scans when available, reducing the `notused` count in plans.
3. **Memory locality:** Index B-trees provide better cache locality than random table access during scans.
4. **Filtering efficiency:** WHERE clause indexes enable early filtering of candidate rows.

5.6.2 Highest-Impact Optimization

The query `student_my_attendance` achieved the highest improvement at **73.89%**, reducing from 0.1012 ms to 0.0264 ms. This query benefited from:

- Large table size (2,400+ `attendance_records`)
- Targeted WHERE clause on `student_id`
- Index completely eliminated the full table scan
- ORDER BY still uses temporary sort, but operates on smaller filtered result set

Similar pattern observed in `student_course_sessions` (72.17% improvement) and `instructor_sessio` (64.99% improvement).

5.6.3 Fair Improvement Queries

Two queries exhibited minimal improvement or regression:

instructor_my_courses and **student_my_courses:** Despite added indexes, these queries showed no or negative improvement. Analysis reveals:

1. **Small result sets:** These queries filter to 1-4 rows
2. **Query planner choice:** For tiny result sets, full scans are sometimes faster than index lookups
3. **Negligible impact:** Difference is < 0.01 ms in either direction
4. **No harm:** Index presence does not degrade performance

5.7 Summary of Indexing Impact

Overall Achievement: The 13-index strategy successfully improved query performance by **39.43%** on average, with 40% of queries achieving excellent improvements (50%+). Most importantly, queries accessing large tables (> 1000 rows) achieved 65–74% speedups by eliminating full table scans. The two-query regression is negligible (<0.01 ms absolute difference) and reflects optimal query planner behavior for trivial result sets.

External Tools and Sources

Tool / Source	Category	Usage
Flask 3.x	Backend framework	API routing, blueprints, request handling
SQLite 3	Database	Local relational database
Werkzeug	Security library	Password hashing (`generate_password_hash`)
Bootstrap 5.3	CSS framework	UI components, tab navigation, responsive layout
Bootstrap JS Bundle	JS library	Modal, tab, and dropdown behaviour
Python `hmac`	Standard library	HMAC-SHA256 session MAC computation
Python `uuid`	Standard library	UUID4 session ID generation
Python `logging`	Standard library	Audit log file writing
Python `queue`	Standard library	Per-client SSE event queues
Jinja2	Templating engine	Server-side HTML shell rendering
Claude (Anthropic)	AI assistant	Code assistance and debugging

Team Member Contributions

Name	Roll No.	Contributions
Rishabh Jogani	23110276	Database Setup, Logging and RBAC
Sidhharth Rajandekar	23110310	Indexing and Benchmarking
Tejas Lohia	23110335	Module-A
Umang Shikarvar	23110301	API management and Testing
Zainab Kapadia	23110373	Full- Stack implementation for the application

All members participated in design discussions, code reviews, and the final video demonstration. Individual contributions above reflect primary ownership of each component.